

Output-Driven Development (ODD): Foundations of Artifact Legitimacy in AI-Native Software Engineering

Yi Fu
ODDFounder
Email: fuyi.it@live.cn

Abstract—Large Language Models (LLMs) make code generation abundant, but they also collapse the traditional control surface of software engineering: changes can be produced faster than humans can review, and responsibility no longer maps cleanly to authorship. We present Output-Driven Development (ODD), an artifact-centric governance workflow for AI-assisted development. ODD shifts approval from diffs to accepted outputs by binding each release artifact to (1) an acceptance contract, (2) an evidence bundle produced by layered checks (tests, adversarial probes, and mutation testing), and (3) a signed seal that records who accepted what and when. This turns acceptance into an auditable operation and focuses human attention on explicit decision points and exceptions. We provide a minimal vignette and an adoption checklist for practitioners.

Practitioner Insights:

- Shift control from code review to artifact acceptance: approve contracts, not diffs.
- Make acceptance auditable: seal each artifact with contract + evidence hashes and a human sign-off.
- Scale verification with layered checks and treat exceptions as first-class events.

Index Terms—Output-Driven Development, AI-Native Software Engineering, Software Governance, Artifact Legitimacy, Acceptance Contracts, Mutation Testing, Provenance, Accountability

I. INTRODUCTION

A. Background: the broken control surface

Recent advances in large language models (LLMs) have fundamentally altered how software is produced [1], [2]. Code generation systems such as Copilot and ChatGPT are no longer auxiliary tools; they increasingly act as *primary producers* of executable artifacts.

However, while generation capability has advanced rapidly, the control structures of software engineering have not evolved accordingly. Traditional methodologies—including Agile, DevOps, Test-Driven Development (TDD), and Model-Driven Engineering (MDE)—implicitly assume that:

- source code is authored by humans,
- reasoning about behavior is mediated through code inspection,
- responsibility can be traced through authorship and commit history.

These assumptions no longer reliably hold in AI-assisted production.

As a result, modern software systems increasingly exhibit a paradoxical condition:

**Code is abundant, yet responsibility is diffuse.
Automation is powerful, yet control is fragile.**

Recent developer surveys suggest that AI-assisted coding has reached critical mass, but verification and accountable acceptance have not kept up. For example, Stack Overflow’s 2024 Developer Survey reports broad use (or planned use) of AI tools alongside a persistent trust gap in AI output [3]. Sonar’s 2026 developer survey highlights what it calls a “verification gap” between low trust and inconsistent verification practices [4]. These signals suggest that the limiting factor is shifting from generation speed to verification and responsible acceptance.

B. The core problem: loss of responsibility anchors

The central challenge introduced by AI-assisted software generation is not correctness alone, but the erosion of *responsibility anchors*.

In conventional workflows, responsibility is implicitly attached to the human author, the review process, or module ownership. In AI-assisted workflows, however:

- code may be generated, revised, and regenerated multiple times,
- generation paths are opaque and non-deterministic,
- multiple agents (human and AI) may contribute asynchronously.

Under such conditions, the question “Who is responsible?” becomes structurally ill-defined.

Existing responses largely attempt to restore control by:

- improving prompt engineering and synthesis-oriented practices [5],
- adding more tests,
- constraining models or sandboxing execution.

While useful, these approaches operate within the code-centric paradigm and therefore fail to address the deeper structural shift.

ODD is not a new prompt recipe, a new testing technique, or a repackaging of Design by Contract (DbC) [6], TDD [7], CI quality gates, or software supply-chain signing [8]–[10]. It is a change in the *control surface*: the primary object of governance becomes the artifact and its acceptance boundary,

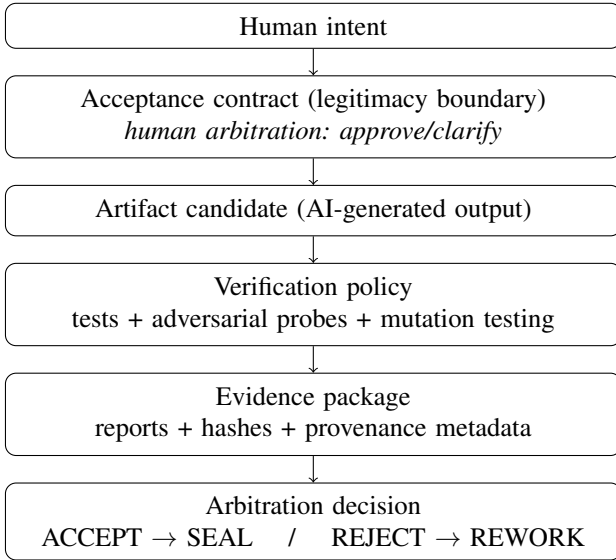


Figure 1. ODD control surface as artifact legitimacy.

and responsibility is anchored in explicit arbitration decisions rather than continuous code review [11].

C. Output-Driven Development (ODD): a paradigm shift

To address the above challenges, we introduce Output-Driven Development (ODD), an artifact-centric methodology for AI-assisted software engineering.

ODD is founded on three core principles:

- 1) **Outputs as first-class entities.** Artifacts are explicitly specified, validated, and audited.
- 2) **Contracts before generation.** Acceptable output spaces are defined prior to any AI execution.
- 3) **Responsibility through arbitration.** Humans are positioned as contract reviewers and final arbiters, not routine implementers.

Rather than attempting to eliminate human involvement, ODD concentrates human attention on irreducible decision points, while delegating routine generation and validation to AI systems.

D. Output-Driven Development (ODD) in one figure

E. Contributions

This paper makes the following contributions:

- 1) **Control-surface reframing.** We define ODD as a shift in software governance: from code/process as the primary control unit to artifact legitimacy as the unit of acceptance.
- 2) **Responsibility anchoring via arbitration.** We make explicit the responsibility shift from authorship and routine code review to contract approval and artifact acceptance at well-defined decision points [11].
- 3) **Operational and formal definitions.** We provide testable definitions of artifact, legitimacy, evidence

package, and sealing, relating sealing to provenance/attestation concepts in software supply chain security [8]–[10].

- 4) **A minimal workflow + vignette.** We outline a lightweight workflow and a concrete vignette showing how ODD can ship a high-risk change without line-by-line code review.
- 5) **Implications and research agenda.** We summarize implications for practitioners and open research directions for contracts, verification economics, and governance under AI-assisted production.

F. Scope

This article focuses on a practical control-surface shift and a pilotable workflow. Empirical validation with production data is left for future work.

II. CORE CONCEPTS

A. Artifact legitimacy as the unit of control

In ODD, the unit of engineering control is not source code or development process steps, but the produced artifact—a verifiable output that satisfies a human need and has direct use-value.

An artifact is *legitimate* when it is accepted under an explicit contract and backed by verifiable evidence (e.g., tests, mutation analysis, and auditable acceptance records). In other words: ODD optimizes for defensibility of outcomes, not readability of implementations.

Key properties:

- **Verifiable:** correctness can be checked against explicit criteria.
- **Need-fulfilling:** it corresponds to an explicit human need (not an internal engineering activity).
- **Use-value:** it can be used to produce real utility.

Intuition. Mature industries do not require customers to observe the entire production process; they rely on rigorous quality control and traceable acceptance.

Operational definition (used in this paper). We treat an artifact as a versioned deliverable that is always referenced by:

- a contract identifier (what is acceptable),
- a verification policy (how it is checked),
- and an evidence package (what was observed when it was checked).

An artifact is *legitimate* when it is accepted under an explicit contract, backed by an evidence bundle produced by verification, and recorded as a seal that binds artifact+contract+evidence at a specific time.

B. Contracts as legitimacy boundaries

A contract in ODD is a precise, machine-checkable specification of acceptable outputs, boundaries, and failure modes. It serves two functions:

- 1) **Constraint definition:** it bounds the acceptable output space before any generation happens.

- 2) **Responsibility anchoring**: acceptance is an explicit, auditable decision at a well-defined boundary.

ODD does not assume the contract is a complete system model; it is a control instrument that makes acceptance legible and scalable.

C. Verification and sealing (high level)

ODD treats verification as a layered pipeline that targets the artifact rather than a transient implementation:

- 1) Contract compliance checks (tests/spec checks)
- 2) Robustness checks (boundary/adversarial probing)
- 3) Test adequacy checks (mutation testing) [12], [13]
- 4) Sealing (recording evidence and freezing the accepted artifact)

The goal is not to claim “the generator is trustworthy,” but to produce an artifact whose acceptance is defensible given explicit criteria and recorded evidence.

Sealing (operational). A seal is an immutable record that (a) identifies the artifact version, (b) records the contract version and verification results, and (c) binds them together with cryptographic hashes/signatures. This is similar in spirit to software supply chain provenance and signing frameworks [8]–[10], but used here as a governance primitive for AI-generated artifacts.

D. Making Sealed(A, s, t) concrete: a seal record sketch

To operationalize the predicate Sealed(A, s, t), ODD uses a *seal record* that binds the artifact, contract, verification policy, and evidence via hashes, and ties acceptance to a human arbitration identity. A minimal sketch is:

```
{
  "seal_id": "SEAL-2026-01-28-0001",
  "timestamp_utc": "2026-01-28T00:00:00Z",
  "arbiter_id": "user:alice (role: contract_approver)",
  "artifact": {
    "id": "payment-charge-endpoint",
    "hash": "sha256:..."
  },
  "contract": {
    "id": "PAYMENTS-GATEWAY-V1",
    "hash": "sha256:..."
  },
  "verification": {
    "policy_id": "VP-HIGH-RISK-V1",
    "results": {
      "tests": "pass",
      "adversarial": "pass",
      "mutation": "pass"
    },
    "evidence_hash": "sha256:..."
  },
  "environment_hash": "sha256:...",
  "signature": {
    "scheme": "sigstore",
    "bundle": "..."
  }
}
```

In practice, seals are appended to a tamper-evident log and audited by verifying the signature and recomputing hashes against the stored evidence and provenance [8]–[10].

E. A minimal vignette: shipping a payment integration without code review

Consider a team integrating a payment gateway. AI can generate the integration quickly, but human review bandwidth is scarce and the risk of silent regressions is high. In ODD, the team does not ask humans to read every line; it asks humans

to approve what “acceptable” means and to accept artifacts only with evidence.

Step 1—Contract (legitimacy boundary). Before generation, the team approves a contract that makes failure modes explicit:

```
contract_id: PAYMENTS-GATEWAY-V1
artifact: payment-charge-endpoint
inputs:
  idempotency_key: required
invariants:
  - never_double_charge
timeouts:
  upstream_timeout_ms: 1000
retries:
  max_attempts: 2
errors:
  - name: timeout
    rule: "no charge may be committed"
  - name: duplicate_idempotency_key
    rule: "return existing transaction id"
audit:
  required_log_fields: [request_id, idempotency_key,
    provider_txn_id]
```

Step 2—Generation + verification (evidence). An AI agent produces implementation and tests. The system then executes the verification policy (unit/integration tests, adversarial probes for edge cases, and mutation testing to check test adequacy).

Step 3—Sealing (acceptance record). If verification passes, the artifact is accepted and sealed with an evidence package, e.g.:

```
{
  "artifact_id": "payment-charge-endpoint",
  "contract_id": "PAYMENTS-GATEWAY-V1",
  "decision": "accept",
  "evidence": {
    "tests": "pass",
    "adversarial_suite": "pass",
    "mutation_testing": "pass"
  },
  "hashes": {
    "contract": "sha256:...",
    "code": "sha256:...",
    "tests": "sha256:..."
  }
}
```

Humans intervene only when the contract is ambiguous, verification fails, or risk policy requires explicit sign-off—not as a routine code-reading bottleneck.

Failure and escalation examples.

- If mutation testing reveals that the test suite is inadequate, the artifact is rejected; the team strengthens tests (or the verification policy) and re-runs the pipeline.
- If the acceptance contract is ambiguous, arbitration clarifies requirements and increments the contract version before any acceptance decision.
- If adversarial probes expose boundary violations, treat the result as a contract violation and reject/rework.

III. ODD IN CONTEXT

ODD is best understood as an axis of control that is orthogonal to code-centric methodologies. It can coexist with existing

practices while changing what is treated as the primary object of governance:

- **Agile/DevOps**: ODD does not change iteration cadence; it changes what is audited and accepted.
- **TDD**: tests remain valuable, but become one layer of evidence rather than the sole specification of intent.
- **MDE**: contracts are not comprehensive models; they are legitimacy boundaries over outputs.
- **Prompt engineering**: prompts may improve generation, but responsibility requires artifact-level evidence.
- **HITL**: humans move from continuous process supervision to decision-point arbitration.

For extended diagrams and detailed comparison tables removed for length, see [Paper_01_ODD_Core_English_Submission_Supplement.md](#).

A. ODD vs. Design by Contract (DbC)

DbC [6] introduced contracts as a way to specify and check module behavior via preconditions, postconditions, and invariants. ODD differs in scope and control surface.

Target of governance. DbC assumes human-authored modular components; ODD assumes AI-generated, frequently re-generated implementations where authorship is diffuse.

Primary question. DbC asks “Is this implementation correct with respect to pre/post conditions?” ODD asks “Is this artifact acceptable under an explicit acceptance boundary, with recorded evidence, at time t ?”

Time and responsibility. DbC checks conditions at run-time; ODD includes sealing—a time-indexed acceptance attestation that freezes a specific artifact+contract+evidence tuple and anchors responsibility in the arbitration decision.

In short, DbC is a specification/verification technique; ODD is a governance paradigm that reallocates responsibility from authorship to arbitration under auditability constraints.

B. Related paradigms and adjacent work

ODD aligns with observations that modern code review is constrained by human understanding bandwidth and provides benefits beyond defect finding [11]. ODD borrows from provenance/attestation work [8]–[10] while shifting the motivation from consumer security to organizational responsibility at scale. The broader context of LLM-based automation in software engineering is surveyed in [14].

IV. IMPLICATIONS OF AN ARTIFACT-CENTRIC CONTROL MODEL

A. Responsibility reallocation in AI-assisted production

In traditional software engineering, responsibility is implicitly anchored in authorship. Code is written, reviewed, and maintained by identifiable individuals or teams, enabling post hoc attribution.

In AI-assisted development, this linkage weakens substantially:

- generation paths are non-deterministic,
- code authorship is diluted across multiple agents,
- implementations are frequently regenerated.

Under such conditions, responsibility can no longer be reliably inferred from process participation.

ODD addresses this issue by reallocating responsibility from authorship to arbitration:

Responsibility in ODD is not derived from who wrote the code, but from who accepted the artifact under a specified contract.

This shift preserves accountability without requiring transparency into AI internals.

B. Contracts as responsibility anchors

In ODD, contracts serve a dual function:

- 1) **Constraint definition**: they define the acceptable space of outputs.
- 2) **Responsibility anchoring**: they establish explicit decision points at which acceptance or rejection occurs.

By requiring human confirmation at contract boundaries—rather than continuous supervision—ODD makes responsibility explicit, auditable, and scalable.

Importantly, contracts in ODD are not assumed to be complete system descriptions. They are control instruments, not ontological models.

C. Verification beyond correctness

Verification in conventional workflows is largely concerned with correctness relative to implementation.

ODD broadens the notion of verification to include:

- contract compliance,
- robustness under perturbation,
- adversarial resistance.

This motivates a layered verification structure in which:

- unit and integration tests assess functional behavior,
- mutation testing evaluates test adequacy [12], [13],
- adversarial testing probes boundary conditions and misuse scenarios.

Crucially, verification targets the artifact, not the implementation.

An implementation may vary;
an accepted artifact must remain defensible.

D. Progressive trust and conditional human intervention

A central design principle of ODD is *progressive trust*. Rather than applying uniform scrutiny to all tasks, ODD dynamically adjusts verification rigor based on:

- estimated complexity,
- safety sensitivity,
- external dependencies.

Human intervention is triggered only when:

- contracts are ambiguous,
- adversarial agents disagree,
- or high-risk conditions are detected.

This approach preserves human authority while preventing cognitive overload.

Human attention is treated as a scarce governance resource.

E. Governance without full transparency

A common objection to AI-assisted systems is the lack of model transparency. ODD deliberately avoids assuming:

- interpretable models,
- explainable generation paths,
- deterministic outputs.

Instead, it adopts a governance strategy based on observable outcomes and documented acceptance decisions. This makes ODD compatible with proprietary models, evolving AI architectures, and heterogeneous toolchains.

Governance is achieved not by introspection, but by controlled acceptance.

F. Risk, failure, and explicit non-goals

ODD does not eliminate risk. It makes risk legible.

Failures in ODD manifest as:

- contract violations,
- failed verification stages,
- unresolved arbitration.

These failure modes are explicit and auditable.

Equally important are ODD's non-goals:

- it does not aim to automate ethical judgment,
- it does not guarantee optimal implementations,
- it does not replace domain expertise.

ODD provides structure, not omniscience.

G. Implications for AI-native organizations

As AI systems become primary producers rather than assistants, organizations face a structural challenge:

How can legitimacy be maintained when production is automated?

ODD suggests an answer:

- legitimacy derives from controlled acceptance,
- authority is exercised through arbitration,
- responsibility is preserved through explicit contracts.

H. Practical adoption checklist (first iteration)

- Pick 1–2 high-value artifact types and define what acceptance means for each.
- Create contract templates that force boundary conditions (timeouts, idempotency, error codes, invariants) to be explicit.
- Define a verification policy per risk level (what tests must run, what adversarial probes are required, what mutation threshold is acceptable).
- Treat sealing as an append-only acceptance record (artifact version, contract version, evidence hashes); require a new seal for any change.
- Make exceptions explicit: when humans override automation, record who decided, why, and what evidence was missing.

V. LIMITATIONS AND FUTURE WORK

ODD is presented as a foundations-and-practice proposal. Its effectiveness depends on (1) the quality and maintainability of acceptance contracts, and (2) the cost of verification (mutation testing can be expensive). We plan empirical validation via pilot deployments and longitudinal case studies.

Future work includes: (i) contract templates and lightweight contract languages, (ii) scalable verification (incremental/sampled/parallel mutation testing), and (iii) tooling for sealing, exception handling, and audit queries.

VI. CONCLUSION

This paper proposes Output-Driven Development (ODD), a software governance paradigm for the AI era.

A. Summary of contributions

- 1) **Reframed the control problem.** AI-assisted development breaks the traditional control surface where human code review served as the final quality gate.
- 2) **Introduced artifact-centric governance.** ODD centers on artifacts—verifiable outputs that satisfy human needs—rather than code.
- 3) **Established contracts as responsibility anchors.** Contracts define what constitutes acceptable output, creating explicit decision points for human acceptance.
- 4) **Proposed verification beyond code review.** ODD replaces routine code review with layered verification (compliance, mutation testing, adversarial probing).
- 5) **Articulated governance without transparency.** Effective governance does not require model interpretability; it requires controlled acceptance and auditable decisions.

B. The core insight

In AI-assisted development, humans should define *what* is acceptable, not supervise *how* it is produced.

C. Implications

If ODD's premises are correct, several implications follow:

- **For practitioners:** skills shift from coding proficiency to contract specification, verification design, and acceptance judgment.
- **For organizations:** workflows must be restructured around artifact pipelines rather than code commits.
- **For researchers:** new research agendas emerge around contract languages, scalable mutation testing, and human-AI collaboration patterns.

D. Subsequent work in the ODD series

While this paper introduces ODD as an artifact-centric paradigm, it does not yet address several questions: contract acquisition, enforcement mechanisms, legitimacy evolution, and context engineering. These are addressed in subsequent papers in the ODD series.

E. Closing remarks

ODD is not a finished methodology. It is a proposal whose validity will be determined by empirical evidence, practical adoption, and the judgment of the engineering community.

APPENDIX A SUPPLEMENTARY MATERIAL

Extended diagrams and detailed comparison tables removed for length are collected in:

- `Paper_01_ODD_Core_English_Submission_Supplement.md`

The detailed technical content—including contract templates, mutation testing configurations, sealing record structures, and implementation notes—is available in the companion technical reference via Zenodo [15].

REFERENCES

- [1] S. Bubeck, V. Chandrasekaran, R. Eldan *et al.*, “Sparks of artificial general intelligence: Early experiments with GPT-4,” arXiv preprint arXiv:2303.12712, 2023. [Online]. Available: <https://arxiv.org/abs/2303.12712>
- [2] M. Chen, J. Tworek, H. Jun *et al.*, “Evaluating large language models trained on code,” arXiv preprint arXiv:2107.03374, 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [3] S. Overflow, “Developer survey 2024,” Online survey, 2024. [Online]. Available: <https://survey.stackoverflow.co/2024/>
- [4] Sonar, “Sonar data reveals critical “verification gap” in ai coding,” Press release, 2026. [Online]. Available: <https://www.sonarsource.com/company/press-releases/sonar-data-reveals-critical-verification-gap-in-ai-coding/>
- [5] J. Austin, A. Odena, M. Nye *et al.*, “Program synthesis with large language models,” arXiv preprint arXiv:2108.07732, 2021. [Online]. Available: <https://arxiv.org/abs/2108.07732>
- [6] B. Meyer, “Applying design by contract,” *Computer*, vol. 25, no. 10, pp. 40–51, 1992.
- [7] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- [8] S. Torres-Arias, H. Afzali, T. Kuppusamy, J. Cappos *et al.*, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1393–1410.
- [9] OpenSSF, “Supply-chain levels for software artifacts (slsa),” Version 1.0, 2023. [Online]. Available: <https://slsa.dev>
- [10] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software signing for everybody,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 2353–2367.
- [11] A. Bacchelli and C. Bird, “Expectations, outcomes, and challenges of modern code review,” in *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 712–721.
- [12] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.
- [13] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, “Hints on test data selection: Help for the practicing programmer,” *Computer*, vol. 11, no. 4, pp. 34–41, 1978.
- [14] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, “Large language models for software engineering: Survey and open problems,” arXiv preprint arXiv:2310.03533, 2023. [Online]. Available: <https://arxiv.org/abs/2310.03533>
- [15] Y. Fu, “ODD Output-Driven Development - A Novel Methodology for AI-Assisted (v1.0),” Zenodo, 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.18207648>